SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or technically

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

HONEY CHAIN: A Blockchain-Based System for Honey Traceability and Smart Beekeeping Management

1. Analyze System Requirements

1.1 System Objectives

HONEY CHAIN is a blockchain-based system designed to improve honey traceability, transparency, and beekeeping management. It tracks honey from production to the final consumer and maintains reliable records of harvesting, processing, packaging, and distribution. The system also uses IoT sensors to monitor hive conditions and helps consumers verify honey authenticity through QR codes.

The main objectives are:

- Track honey throughout the supply chain.
- Maintain secure and tamper-resistant records.
- Monitor hive temperature, humidity, and health conditions.
- Enable QR-code-based product verification.
- Improve trust between beekeepers, distributors, and consumers.

1.2 Functional Requirements

Functional requirements describe the main operations performed by the system.

- User Management: Register and authenticate beekeepers, processors, distributors, and consumers.
- Hive Monitoring: Collect temperature, humidity, and other available sensor readings.
- Harvest Management: Record harvest dates, quantities, and batch identification numbers.
- Blockchain Traceability: Store important supply-chain events on the blockchain.
- Quality Control: Record honey testing results and quality certificates.
- QR Code Verification: Allow consumers to view a product's traceability history.

- Notifications: Send alerts for abnormal hive conditions and important supply-chain events.
- Report Generation: Generate production and quality reports.

1.3 Non-Functional Requirements

- Non-functional requirements define the quality and performance expected from the system.
- Scalability: Support increasing numbers of users, hives, and honey batches.
- Security: Protect sensitive information and prevent unauthorized access.
- Performance: Provide quick access to product details and verification results.
- Reliability: Maintain accurate records and minimize data loss.
- Availability: Ensure the system remains accessible when required.

Maintainability: Allow individual components to be updated without affecting the entire system.

2. Select a Suitable Software Architecture

The most suitable architecture for HONEY CHAIN is a Hybrid Layered Architecture integrated with Blockchain and IoT. This architecture combines the separation of responsibilities offered by layered architecture with the secure recordkeeping of blockchain and the automated monitoring capabilities of IoT devices.

The system is divided into different layers, including the presentation layer, application layer, business services layer, data storage layer, and blockchain layer. Each layer performs a specific function and communicates with other layers through APIs and defined interfaces.

The reasons for selecting this architecture are:

- Layered Architecture: Separates the user interface, business logic, and data management.
- Client–Server Architecture: Enables access through web and mobile applications.

- Blockchain Technology: Maintains tamper-evident records of honey transactions.
- IoT Integration: Supports automatic hive monitoring.
- API Integration: Allows communication with external laboratories and notification services.

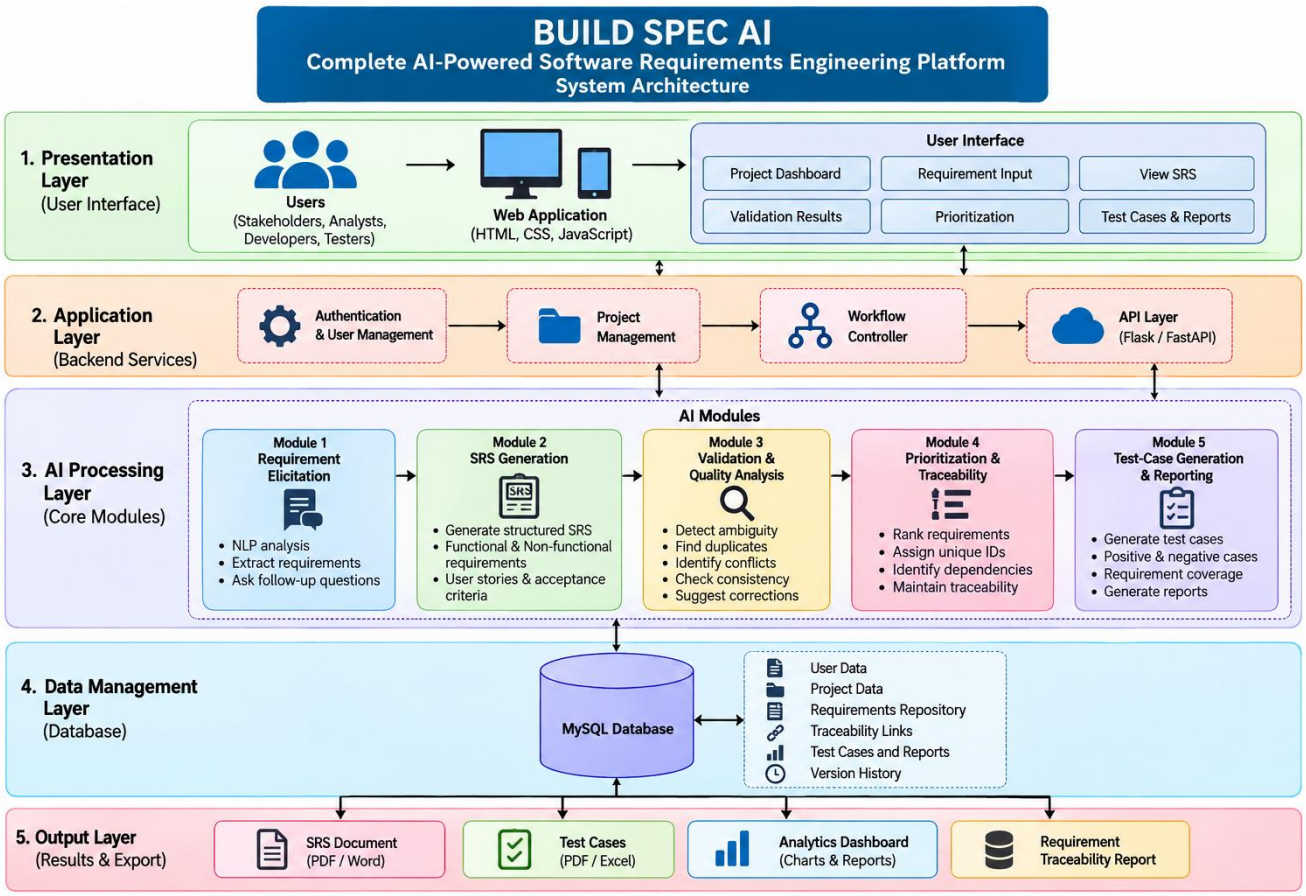
This combination provides a flexible, secure, and maintainable solution for honey traceability and smart beekeeping management.

3. Draw the Software Architecture Diagram

The software architecture diagram of HONEY CHAIN illustrates the overall structure and interaction of the system's major components. It consists of the presentation layer, application layer, business services layer, data storage layer, and blockchain layer. The presentation layer provides interfaces for beekeepers, processors, distributors, and consumers, while the application layer manages user authentication, access control, and request validation.

The business services layer handles hive monitoring, honey harvesting, quality control, QR-code generation, traceability, and notifications. The data storage layer maintains user details, sensor readings, product information, and quality certificates. The blockchain layer securely records important supply-chain events using smart contracts, ensuring that recorded information is tamper-evident and traceable. External systems, including IoT sensors, quality-testing laboratories, logistics providers, and email/SMS services, communicate with the platform through appropriate interfaces.

Overall, this architecture provides a secure, scalable, and maintainable solution for tracking honey from production to the final consumer. It improves supply-chain transparency, supports efficient beekeeping management, and enables consumers to verify product information by scanning QR codes.



The architecture diagram illustrates the major components and their communication paths. Users access the system through the presentation layer, while the application layer validates requests and directs them to the appropriate business service. The database stores operational information, and the blockchain maintains verifiable records of important supply-chain events. External devices and systems communicate through suitable APIs and IoT protocols.

4. Explain Components and Their Interactions

The architecture contains several components that work together to ensure secure and efficient honey traceability.

Major components and their responsibilities:

- Presentation Layer: Provides interfaces for beekeepers, processors, distributors, and consumers.
- Application Layer: Handles incoming requests, authentication, and access permissions.

- Hive Monitoring Service: Collects sensor readings and generates alerts when configured thresholds are exceeded.
- Harvest Management Service: Records harvesting details, quantities, dates, and batch IDs.
- Quality Control Service: Stores laboratory results and quality certificates.
- Blockchain Service: Records validated supply-chain events and executes smart contracts.
- Database: Stores user profiles, product metadata, sensor readings, and operational records
- QR Traceability Service: Generates QR codes and retrieves product histories.
- Notification Service: Sends alerts and updates through email or SMS.

System workflow

The workflow begins when a beekeeper registers a hive and records honey harvesting information. A unique batch ID is generated, and the honey is sent for quality testing. Important events, such as harvesting, testing, packaging, and distribution, are validated and recorded on the blockchain. A QR code is then associated with the packaged product. When a consumer scans the QR code, the system retrieves the available traceability history and displays the product's origin and journey.

For example, a beekeeper harvests 50 kg of honey and creates batch HC001. After quality testing and packaging, the relevant records are linked to this batch. A consumer scanning the QR code can view the recorded information and verify whether it matches the product's registered history.

5. Discuss Quality Attributes

Quality attributes determine how effectively the system operates under different conditions. HONEY CHAIN addresses these attributes through its layered structure, blockchain integration, and modular services.

- Scalability: Modular services, database indexing, and additional application instances help the system support more users and honey batches.

- Security: HTTPS, authentication, role-based access control, and blockchain validation protect data and transactions.
- Performance: Database indexing and caching provide quick access to frequently requested product information.
- Reliability: Database backups, transaction validation, and replicated blockchain nodes reduce the risk of data loss.
- Maintainability: Separate layers and independent services simplify debugging, testing, and software updates.
- Availability: Redundant application instances and recovery procedures help minimize service interruptions.

However, blockchain alone cannot guarantee that the original data is correct. Sensor calibration, authenticated data entry, and independent quality testing are necessary to improve trustworthiness.

6. Justify Architectural Decisions

The hybrid layered architecture is selected because HONEY CHAIN requires conventional data management as well as secure and transparent supply-chain tracking. The layered approach separates responsibilities, blockchain supports tamper-evident records, and IoT devices provide automated hive monitoring. Using a database for frequently accessed information and blockchain for important traceability events also helps balance performance and data integrity.

The main advantages of this architecture are:

- Improved transparency and traceability of honey products.
- Separation of responsibilities for easier maintenance.
- Integration with IoT devices and external systems through APIs.
- Support for future expansion without redesigning the complete system.
- Better auditing of important supply-chain transactions.

Some trade-offs must also be considered. Blockchain introduces transaction costs and additional infrastructure requirements. IoT devices need regular maintenance, and multiple services require proper coordination. These challenges can be managed through a permissioned blockchain, efficient data storage, and clear communication interfaces.

In the future, HONEY CHAIN can be enhanced with AI-based hive-health prediction, mobile applications, logistics integration, digital quality certificates, and advanced production analytics.

7. Conclusion

HONEY CHAIN combines layered architecture, blockchain technology, and IoT integration to develop a secure and transparent honey traceability system. It supports hive monitoring, harvest management, quality control, and QR-code-based consumer verification. Its modular design improves scalability, security, performance, reliability, and maintainability. Therefore, the proposed architecture provides a suitable foundation for smart beekeeping management and trustworthy honey supply-chain operations.